NTU-DAVID-RAG

A Retrieval-Augmented Generation System for
Economics Research Papers and Fortran Model Code

Technology, Pipeline, and RAGAS Evaluation

June 9, 2026

Abstract

NTU-DAVID-RAG is a retrieval-augmented generation (RAG) system that lets researchers ask natural-language questions over a corpus of computational-economics papers and their accompanying Fortran source code. The system pairs a structured ingestion pipeline (PDF layout parsing, multimodal enrichment, section-aware chunking, and dense embedding) with a hybrid retrieval pipeline (sparse BM25 + dense vectors, reciprocal-rank fusion, and cross-encoder reranking) feeding a grounded, locally hosted large language model. This report documents the technology stack, explains the end-to-end data flow for both ingestion and querying, describes the containerized deployment, and reports reference-free quality measurements obtained with the RAGAS framework across several judge models.

Contents

1 System Overview

The system answers two kinds of questions over a private, per-user corpus: *local* questions about a single paper (a definition, an equation, a table value, a Fortran routine) and *global* questions that compare or synthesize across papers ("which papers use value-function iteration?"). To serve both well it combines lexical and semantic retrieval, a paper-level profile index for cross-document reasoning, and strict context-grounding at generation time.

The corpus is unusual for RAG in three ways that shape every design decision:

- **Heavy mathematics.** Content is dense in LaTeX equations and transition matrices, so extraction preserves equations and tables as first-class, separately describable objects.
- **Code alongside prose.** Each paper may ship a Fortran implementation; the system parses it into routine-level units with a call graph so questions about `rw_decision` or value-function iteration can be answered from code, not just text.
- **Exact identifiers matter.** Variable and subroutine names must match literally, which is why a sparse BM25 stage runs alongside dense vectors rather than relying on embeddings alone.

2 Technology Stack

Table 1 summarizes the components and the role each plays in the pipeline. The default deployment runs entirely on self-hosted infrastructure; all model inference (embedding, reranking, generation, and the RAGAS judge) can be served locally, with optional hosted judges used only for evaluation.

Stage	Technology	Role
PDF extraction	MinerU (GPU)	Convert PDF to Markdown + structured content list (text, images, equations, tables)
Multimodal enrich.	Local LLM (`gemma4:31b` via Ollama)	Extract title/authors; describe figures, tables, and equations for indexing
Code digestion	Regex parser + LLM	Split Fortran into `PROGRAM/MODULE/SUBROUTINE/FUNCTION` units with a call graph
Chunking	Custom, section-aware	384-token chunks, 64-token overlap, hierarchical section path prepended
Embeddings	`gte-Qwen2-7B-instruct` (3584-dim)	Asymmetric query/document vectors for dense retrieval
Vector store	Qdrant (on-disk, per-user)	Cosine search over `rag_embeddings` and `paper_profiles`
Sparse retrieval	BM25 (`rank-bm25`)	Lexical match; tokenizer preserves Fortran identifiers
Fusion	Reciprocal Rank Fusion (k=60)	Merge sparse + dense ranked lists
Reranking	`bge-reranker-v2-m3` cross-encoder	Score fused candidates against the original query
Generation	Local LLM (`gemma4:31b`)	Grounded answer with citations to paper titles
Serving	Flask + Gunicorn, Nginx, PostgreSQL	API, reverse proxy, auth/conversation store
Orchestration	Docker Compose	Multi-service stack with optional single-GPU mode

Table 1: Component technology stack and the role of each stage.

3 End-to-End Data Flow

Figure 1 shows the two halves of the system: an *ingestion* pipeline that builds the knowledge base, and a *query* pipeline that answers questions from it. The two halves meet at the Qdrant vector database.

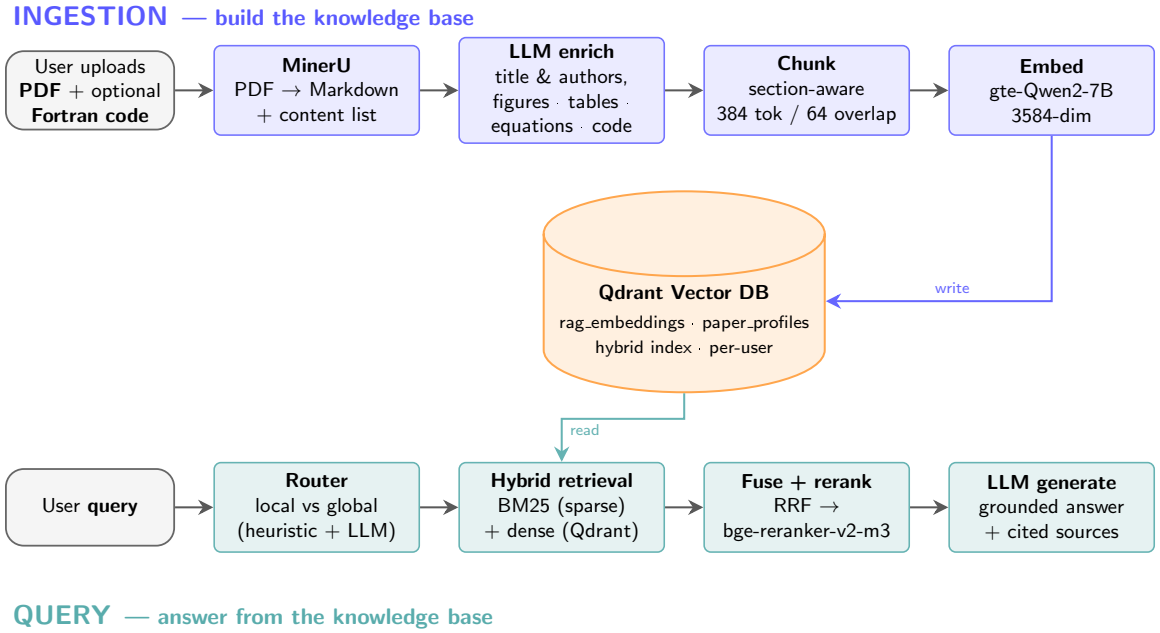

Figure 1: End-to-end RAG pipeline. Ingestion (top) writes embedded, section-aware chunks into Qdrant; querying (bottom) reads from Qdrant via hybrid retrieval, fuses and reranks candidates, then generates a grounded answer.

3.1 Ingestion: building the knowledge base

A user uploads a PDF and, optionally, a Fortran source file. Ingestion proceeds in five stages:

1. **Layout parsing (MinerU).** The PDF is converted to Markdown plus a *content list* that tags every block as text, image, equation, or table, with page indices and reading order preserved.
2. **Enrichment.** A local LLM extracts the paper title and authors and writes natural-language descriptions of figures, tables, and equations so that non-text objects become retrievable. Bibliography after "References" is stripped.
3. **Code digestion (optional).** Fortran is parsed into routine-level units; each unit is summarized for its economic purpose, inputs/outputs, algorithm, and call relationships.
4. **Chunking.** Manuscript text is split on section boundaries into 384-token chunks with 64-token overlap; the hierarchical section path (e.g. *IV. Calibration > D. Rate of Return Process*) is prepended to every chunk so retrieval keeps local context. Each image/equation/table and each Fortran unit becomes its own document.
5. **Embedding and indexing.** All chunks are embedded with `gte-Qwen2-7B` and upserted into Qdrant. In parallel, a per-paper *profile* (research question, methods, key equations, concepts) is embedded into a separate `paper_profiles` collection that powers global, cross-document queries.

3.2 Querying: answering from the knowledge base

1. **Routing.** A heuristic-plus-LLM router classifies the query as *local* or *global* and extracts focus concepts. Global queries first retrieve candidate papers from the profile index, then retrieve within each candidate.

2. **Hybrid retrieval.** The query is embedded for dense cosine search in Qdrant and, in parallel, scored by BM25. Each returns its top candidates.
3. **Fusion and reranking.** The two ranked lists are merged with Reciprocal Rank Fusion (k=60); the fused set is reranked by the `bge-reranker-v2-m3` cross-encoder against the *original* user query, and paper-diversity is enforced so results are not dominated by a single document.
4. **Context assembly and generation.** The top passages are assembled into a structured context (grouped by paper, with equations/tables/code rendered appropriately) and passed to the generation LLM, which is instructed to answer only from the supplied context and cite papers by title. Conversation history is compacted when the prompt would exceed the context budget.

4 Formal Description of the Pipeline

This section makes the retrieval and ranking steps of the query pipeline (Section 3.2) precise. Let the indexed corpus be a set of documents $\mathcal{D} = \{d_1, \ldots, d_N\}$, where each d_i is a chunk, an enriched figure/equation/table description, or a Fortran unit, and let q denote the user query.

4.1 Chunking and token budgeting

Manuscript text is segmented into chunks of at most $L_{\max} = 384$ tokens with an overlap of $L_{\text{ov}} = 64$ tokens, where token counts use the lightweight word-based estimator

$$\widehat{\tau}(t) \;=\; \big\lfloor 1.33 \cdot |\text{words}(t)| \big\rfloor, \tag{1}$$

i.e. roughly 1.33 tokens per whitespace-delimited word. A chunk is flushed when appending the next block b would violate $\widehat{\tau}(\text{chunk}) + \widehat{\tau}(b) > L_{\max}$; the trailing text whose estimated length fits within L_{ov} is carried into the next chunk. Overlap never crosses a section boundary, and the hierarchical section path is prepended to each chunk before embedding.

4.2 Asymmetric embedding

A single encoder $E(\cdot)$ (`gte-Qwen2-7B-instruct`, dim = 3584) maps text to vectors, but queries and documents are encoded *asymmetrically* through distinct instruction prefixes. With

π_q = "Instruct: Given a web search query, retrieve relevant passages that answer the query \n Query: ",

the document and query embeddings are the L_2-normalized encoder outputs

$$\mathbf{v}_d = \frac{E(\pi_d \oplus d)}{\|E(\pi_d \oplus d)\|_2}, \qquad \mathbf{q} = \frac{E(\pi_q \oplus q)}{\|E(\pi_q \oplus q)\|_2}, \qquad \mathbf{v}_d, \mathbf{q} \in \mathbb{S}^{3583}. \tag{2}$$

Because both vectors lie on the unit hypersphere, inner product, cosine similarity, and (the negative of) squared Euclidean distance induce the *same* ranking.

4.3 Dense retrieval

Qdrant scores candidates by cosine similarity, which for unit vectors reduces to the dot product

$$s_{\text{dense}}(q, d) \;=\; \cos\big(\mathbf{q}, \mathbf{v}_d\big) \;=\; \mathbf{q}^\top \mathbf{v}_d. \tag{3}$$

For backends that report a cosine *distance* $\delta \in [0, 2]$ the similarity is recovered as $s_{\text{dense}} = 1 - \delta$. The dense stage returns the top $k_{\text{dense}} = 50$ documents by s_{dense}.

4.4 Sparse retrieval (BM25)

In parallel, lexical relevance is scored with Okapi BM25 over a tokenization that deliberately preserves code-style identifiers (lower-cased alphanumerics plus _ and ., with hyphenated compounds such as `bge-large` kept intact). For query terms $t \in q$,

$$s_{\mathrm{bm25}}(q,d) \;=\; \sum_{t\in q} \mathrm{IDF}(t)\cdot \frac{f(t,d)\,(k_1+1)}{f(t,d)+k_1\Big(1-b+b\,\dfrac{|d|}{\overline{|d|}}\Big)}, \qquad \mathrm{IDF}(t)=\ln\frac{N-n_t+0.5}{n_t+0.5}+1, \quad (4)$$

where $f(t,d)$ is the frequency of t in d, n_t the number of documents containing t, $|d|$ the document length, $\overline{|d|}$ the mean length, and the library defaults $k_1 = 1.5$, $b = 0.75$ are used. Documents with $s_{\mathrm{bm25}} \le 0$ are discarded; the stage returns the top $k_{\mathrm{sparse}} = 50$.

4.5 Reciprocal-rank fusion

The dense and sparse result lists are merged by rank rather than by raw score, which sidesteps the incommensurability of $s_{\mathrm{dense}} \in [-1,1]$ and the unbounded s_{bm25}. Let $r_\ell(d) \in \{1,2,\dots\}$ be the rank of d in list $\ell \in \{\mathrm{dense},\mathrm{sparse}\}$ (omitted lists contribute nothing). With $k = 60$,

$$s_{\mathrm{rrf}}(d) \;=\; \sum_{\ell\in\{\mathrm{dense},\,\mathrm{sparse}\}} \frac{1}{k+r_\ell(d)}. \quad (5)$$

The additive constant k damps the influence of very high ranks so that a document need not top *both* lists to score well; agreement across the two retrievers is rewarded. The top 50 fused candidates, $\mathcal{C} = \arg\mathrm{top}_{50}\ s_{\mathrm{rrf}}$, proceed to reranking.

4.6 Cross-encoder reranking

Fusion uses only ranks; the reranker restores fine-grained query–document interaction by jointly encoding the pair with `bge-reranker-v2-m3`:

$$s_{\mathrm{rerank}}(q,d) \;=\; \mathrm{CE}\big(q \,\|\, d\big), \qquad d \in \mathcal{C}. \quad (6)$$

Unlike the bi-encoder of the dense stage, CE attends over the query and the candidate together, so it can verify exact identifier and equation matches that a single pooled vector blurs. Candidates are sorted by s_{rerank}.

4.7 Paper-diversity selection

To prevent a single document from monopolizing the context, the reranked list is reduced to the final $k_{\mathrm{final}} = 20$ passages by round-robin selection across source papers. Partition the reranked candidates by paper title into groups $G_1,\dots,G_M$, each kept in reranked order; the selection takes the j-th best passage of every paper before any paper's $(j{+}1)$-th:

$$\mathcal{R} \;=\; \big[\, G_p[j] \;:\; j = 0,1,2,\dots,\;\; p = 1,\dots,M \,\big] \quad \text{truncated to } k_{\mathrm{final}}, \quad (7)$$

skipping exhausted groups. The selected chunks are then expanded with their immediate neighbors (sequence index ± 1 within the same paper), each neighbor inheriting a slightly penalized score $s_{\mathrm{rerank}} - 0.01$ so originals sort ahead of context-only additions.

4.8 Global (cross-document) routing

A query is first classified as *local* or *global*. A case-insensitive regex $H(q) \in \{\text{global}, \varnothing\}$ fires on comparative phrasing ("which papers", "compare", "differ", "in common", ...); when it does not fire, an LLM classifier returns a scope and 2–5 focus concepts. The heuristic takes precedence, and the scope defaults to local:

$$\text{scope}(q) = \begin{cases} \text{global}, & H(q) = \text{global}, \\ \text{LLM}_{\text{scope}}(q), & \text{otherwise}. \end{cases} \tag{8}$$

A global query first retrieves the top 6 papers from the `paper_profiles` collection using the concept-enriched query $q' = q \,\|\, \text{"Concepts: "} \,\|\, \text{concepts}$, then runs the hybrid pipeline above *within* each selected paper (drawing 4 chunks per paper) before a final rerank, so cross-document answers stay grounded in per-paper evidence.

5 Deployment Architecture

The system ships as a Docker Compose stack (Figure 2). A public Nginx reverse proxy fronts the `rag-web` Flask application, which orchestrates ingestion and querying. GPU-bound model inference is isolated in a dedicated `embedding-server` (embeddings + reranking) and an Ollama LLM service; PostgreSQL stores users, sessions, and conversations; and each user gets an isolated on-disk Qdrant collection set. A single-GPU override swaps in a smaller embedder (`embeddinggemma-300m`) and an in-stack quantized LLM so the whole stack fits on one card.

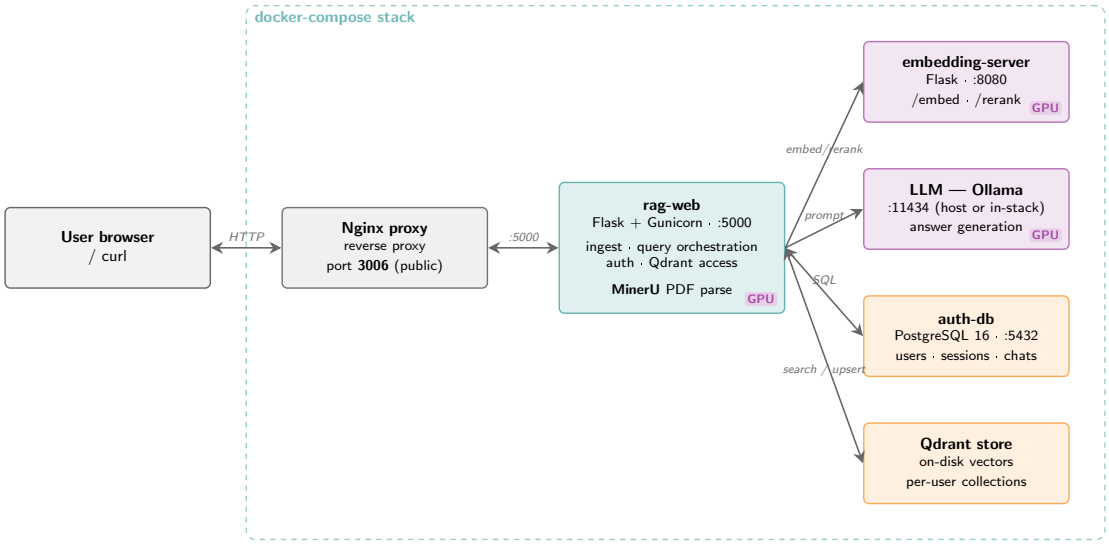

Figure 2: Containerized service topology. `rag-web` is the orchestration hub; GPU services (badged) handle embedding/reranking, PDF parsing, and generation; PostgreSQL and per-user Qdrant provide durable state.

6 Evaluation Methodology (RAGAS)

Quality is measured with RAGAS, a *reference-free* evaluation framework: it scores answers without gold-standard references by using an LLM judge to inspect the question, the generated answer, and the retrieved contexts. Two metrics are reported:

- **Faithfulness** — the fraction of claims in the answer that are directly supported by the retrieved context. It penalizes any statement the context does not entail (a proxy for hallucination).
- **Context Precision (without reference)** — whether the retrieved passages that were used are actually relevant to the question, i.e. the signal-to-noise ratio of retrieval as judged by the LLM.

The evaluation harness (`RAG_quality_test/rag_ragas_eval.py`) issues a fixed question set against the live RAG API, captures each answer together with its retrieved contexts, and scores every item with a configurable judge (*local* Ollama, *OpenAI*, or *Gemini*). The question set is designed so that each item stresses a distinct facet of the pipeline: single-paper factual and methodology retrieval, cross-paper comparative reasoning (the global/profile path), Fortran code retrieval, LaTeX equation retrieval, table/numeric retrieval, and an out-of-scope robustness probe whose correct answer is "none of the papers."

7 Results

Table 2 summarizes five evaluation runs that vary the judge model, question-set size, and `top_k`. The corpus contained 5–6 papers depending on the run.

Date	#Q	Judge model	top_k	Faithfulness	Context Prec.
2026-04-18	1	openai / gpt-5.4	10	0.333	0.700
2026-04-18	30	gemini / gemini-2.5-flash	10	**0.220**	**0.344**
2026-04-18	30	openai / gpt-4o-2024-08-06	10	0.216	0.289
2026-04-20	30	local / gemma4:31b	10	0.125	0.308
2026-05-12	10	local / gemma4:31b	5	0.235	0.267

Table 2: RAGAS aggregate means across runs. The single-question run is a smoke test. The two hosted judges (Gemini, GPT-4o) agree closely on faithfulness (≈ 0.22); the local judge is harsher and far slower.

Headline run. The most representative run is the 30-question, Gemini-judged evaluation (`top_k`=10). Its per-question scores are shown in Figure 3. Mean faithfulness is 0.220 (median 0.218) and mean context precision is 0.344 (median 0.000). Per-question latency averaged 82.8 s (median 75.2 s, range 43–172 s), reflecting the cost of local generation over long, math-heavy contexts.

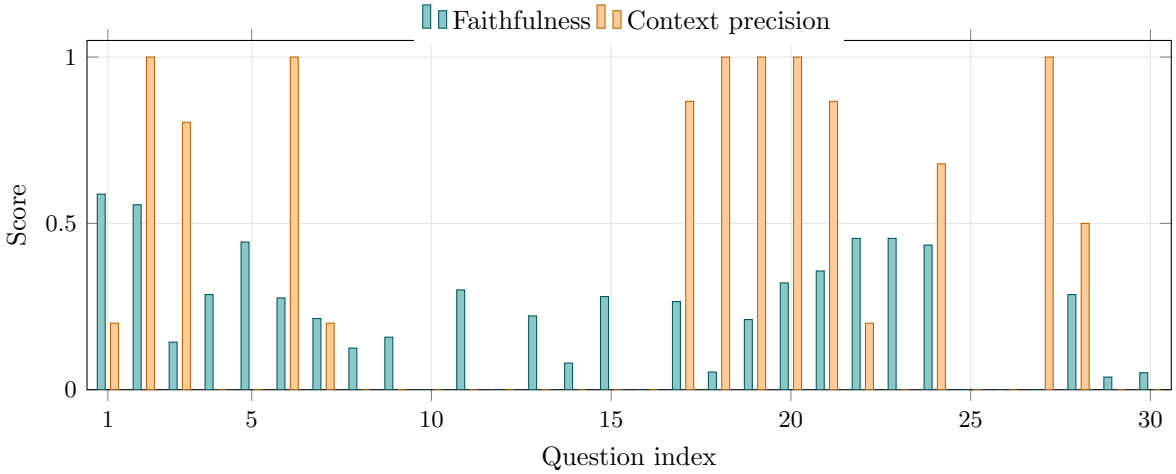

Figure 3: Per-question RAGAS scores for the 30-question Gemini-judged run. Context precision is strongly bimodal: 9/30 questions score ≥ 0.8 while 16/30 score 0 — retrieval is either on-target or off-target, rarely in between.

7.1 Interpretation

- **Faithfulness scores are conservative by construction.** RAGAS decomposes an answer into atomic claims and requires each to be entailed by the retrieved context. Answers in this domain are dense with LaTeX, matrices, and derived statements; a judge that cannot verify a transcribed equation token-for-token marks the claim unsupported, deflating the score. The close agreement between two independent hosted judges (0.220 vs. 0.216) indicates the *ranking* is stable even if the absolute level is pessimistic.
- **Context precision is bimodal** (Figure 3): retrieval tends to either surface the right passages or miss them, with little middle ground. The misses cluster on equation/table/numeric questions, where the answer hinges on one specific block that must be retrieved verbatim — the motivation for the hybrid BM25 stage and the multimodal equation/table indexing.
- **Judge choice matters.** The local `gemma4:31b` judge is both harsher on faithfulness (0.125) and dramatically slower (264 minutes for 30 questions vs. ∼53–71 minutes for hosted judges), making it useful for offline regression but unsuitable for fast iteration.
- **Latency is generation-bound.** End-to-end time is dominated by local LLM generation over long contexts, not by retrieval; longer answers (equation/derivation questions) sit at the high end of the latency range.

7.2 Two-GPU vs. single-GPU deployment (per-question)

To quantify what single-GPU mode costs (or gains), the same 10-question set was run against both deployments with the generator, judge, and `top_k` held fixed (`gemma4:31b` generation, local `gemma4:31b` judge, `top_k`=5). The only differences are the GPU count and the embedder: the default two-GPU stack runs the 7B `gte-Qwen2-7B` embedder (3584-dim) on a separate card from the generator, while the single-GPU stack collapses every component onto one card with the lighter `bge-large-en-v1.5` embedder (1024-dim).[1] The comparison therefore isolates the effect of moving to a single card.

Table 3 reports the per-question scores and their difference (Δ = single-GPU − two-GPU); Figure 4 visualizes the same deltas.

[1]The single-GPU override's default embedder, `embeddinggemma-300m`, is gated on Hugging Face; the evaluated run substituted the non-gated `bge-large-en-v1.5`.

		2-GPU (gte-Qwen)		1-GPU (bge-large)		Difference	
Q	**Facet**	F	CP	F	CP	ΔF	ΔCP
1	single-paper factual	0.13	0.00	0.38	1.00	+0.25	+1.00
2	single-paper factual	0.67	0.00	0.36	0.33	−0.30	+0.33
3	methodology	0.06	1.00	0.88	1.00	+0.82	0.00
4	cross-paper comparative	0.00	0.00	0.30	0.00	+0.30	0.00
5	cross-paper comparative	0.00	0.00	0.00	0.00	0.00	0.00
6	cross-paper comparative	0.00	0.00	0.00	0.00	0.00	0.00
7	Fortran / code	1.00	0.83	0.36	1.00	−0.64	+0.17
8	equation / formula	0.50	0.00	0.71	1.00	+0.21	+1.00
9	table / numeric	0.00	0.83	0.00	0.00	0.00	−0.83
10	robustness (out-of-scope)	0.00	0.00	0.00	0.00	0.00	0.00
Mean		0.235	0.267	0.299	0.433	**+0.064**	**+0.166**

Table 3: Per-question RAGAS scores for the two-GPU and single-GPU deployments (F = faithfulness, CP = context precision) with the single-GPU − two-GPU difference. Generator, judge, question set, and `top_k` are identical; only the embedder and GPU count differ. The single-GPU stack improves both means.

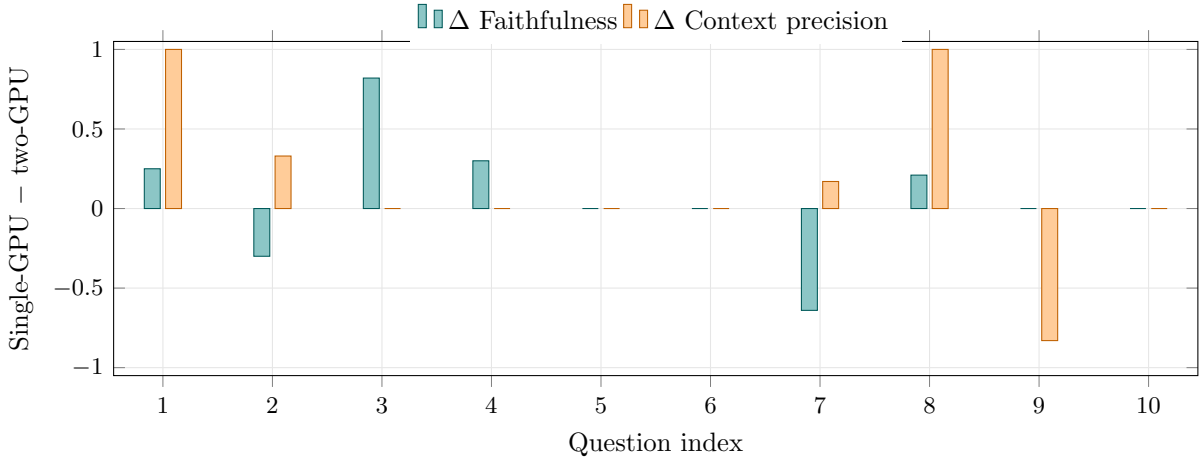

Figure 4: Per-question change from the two-GPU to the single-GPU deployment (positive = single-GPU better). Both means rise; gains concentrate on factual and equation retrieval (Q1, Q8, which jump from 0 to 1.0 context precision), while the two regressions are exact-identifier retrieval (Q7, the Fortran `rw_decision` routine) and exact-table retrieval (Q9).

Interpretation. Counter-intuitively, collapsing onto a single GPU *improves* both aggregate metrics (+0.064 faithfulness, +0.166 context precision). The gain is driven by the embedder: `bge-large` surfaces the relevant passage on factual and equation questions where the larger `gte-Qwen` embedder missed (Q1 and Q8 move from 0 to perfect context precision). The two regressions are both exact-block retrieval — the code-pretrained `gte-Qwen` ranked the precise Fortran routine into the top contexts (Q7) and retrieved the target table (Q9), whereas the prose-oriented `bge-large` did not — reinforcing exact identifier/table retrieval as the highest-value improvement target. Cross-paper comparison (Q5–Q6) and the out-of-scope probe (Q10) score zero on *both* deployments, confirming these are pipeline-level gaps independent of the embedder or GPU budget. For reference, a single-GPU run with the lighter `gemma3:12b-int4` generator in place of `gemma4:31b` reached the same context precision (0.450) at lower faithfulness (0.216), indicating that the embedder governs retrieval precision while the generator governs

faithfulness.

7.3 Threats to validity

These are reference-free, LLM-judged measurements on a small (10–30 question) set over a 5–6 paper corpus, so absolute values should be read as relative signals rather than benchmarks. Faithfulness in particular under-credits correct-but-hard-to-verify mathematical content. The intended use is regression tracking and *controlled* A/B comparisons — such as the two-GPU vs. single-GPU study above, which fixes the judge, question set, generator, and `top_k` so that only the deployment varies — rather than absolute cross-benchmark claims.

8 Conclusion

NTU-DAVID-RAG combines structure-aware ingestion (layout parsing, multimodal enrichment, code digestion, section-aware chunking) with a hybrid retrieval stack (BM25 + dense + RRF + cross-encoder reranking) and a grounded local LLM, all deployed as an isolated, per-user Docker stack. RAGAS evaluation shows stable, judge-agnostic faithfulness around 0.22 and a bimodal context-precision profile that pinpoints equation/table retrieval as the highest-value target for further work — improving exact-block retrieval for mathematical content is expected to lift both metrics simultaneously.